



Πανεπιστήμιο Δυτικής Αττικής

Μοντελοποίηση και Προγραμματισμός Περιορισμών

Ευθύμιος Μάντης, 13ο εξάμηνο, 19390129

Τμήμα Μηχανικών

Μηχανικών Πληροφορικής και Υπολογιστών

Εργασία Εξαμήνου

Μέρος Α' – Τεχνική Υλοποίησης

A1. Ορισμός Οντοτήτων & Δεδομένων

Κώδικας:

```
students = ["S1", "S2", "S3", "S4", "S5"]  
lectures = ["Math", "Physics", "Programming", "Networking", "CSP", "Cryptography"]  
times = ["Mon_9", "Mon_11", "Tue_9", "Tue_11", "Wed_9", "Thu_11", "Fri_9"]  
rooms = ["R1", "R2", "R3", "R4"]
```

Ανάλυση

Το πρόβλημα περιλαμβάνει:

- σύνολο φοιτητών,
- σύνολο μαθημάτων,
- διακριτές χρονικές περιόδους,
- και διαθέσιμες αίθουσες.

Η μοντελοποίηση βασίζεται σε διακριτά σύνολα, κάτι που καθιστά το πρόβλημα κατάλληλο για CSP.

A2. Μεταβλητές Απόφασης

Χρονική Ανάθεση Μαθήματος

Κώδικας:

```
x[l, t] = model.NewBoolVar(f"x[{l}], {t}")) (το μάθημα l γίνεται στην ώρα t)
```

Κάθε μάθημα πρέπει να ανατεθεί σε ακριβώς μία ώρα:

```
model.Add(sum(x[l, t] for t in times) == 1)
```

Ανάθεση Αίθουσας

Κώδικας:

```
y[l, r] = model.NewBoolVar(f"y[{l}], {r}")) (το μάθημα l γίνεται στην αίθουσα r)
```

Περιορισμός:

```
model.Add(sum(y[l, r] for r in rooms) == 1)
```

Ακέραια Μεταβλητή Χρόνου

Κώδικας

```
lec_time[l] = model.NewIntVar(0, len(times)-1, f"time[{l}]")  
model.Add(sum(time_index[t] * x[l, t] for t in times) == lec_time[l])
```

Ανάλυση

Η μεταβλητή αυτή:

- επιτρέπει χρονικές συγκρίσεις (<, >),
- χρησιμοποιείται για προαπαιτούμενα και διάλεξη → εργαστήριο

A3. Περιορισμοί (Hard Constraints)

Μοναδική χρήση αίθουσας

Κώδικας

```
z[l,r,t] = model.NewBoolVar(...)  
model.AddBoolAnd([x[l,t], y[l,r]]).OnlyEnforceIf(z[l,r,t])  
model.Add(sum(z[l,r,t] for l in lectures) <= 1)
```

Ανάλυση

Η βοηθητική μεταβλητή z:

- συνδέει ώρα και αίθουσα,
- αποτρέπει ταυτόχρονη χρήση της ίδιας αίθουσας.

Χωρητικότητα αιθουσών

Κώδικας

```
model.Add(y[l, r] * needed <= room_capacity[r])
```

Ανάλυση

Αποκλείει λύσεις όπου:

- μικρές αίθουσες φιλοξενούν μεγάλα μαθήματα.

Διαθεσιμότητα διδασκόντων

Κώδικας

```
model.Add(sum(x[l, t] for t in teacher_availability[l]) == 1)
```

Ανάλυση

Ο καθηγητής:

- πρέπει να είναι διαθέσιμος την ώρα του μαθήματος.

Αποφυγή συγκρούσεων φοιτητών

Κώδικας

```
model.Add(sum(x[l, t] for l in enrolled) <= 1)
```

Ανάλυση

Εξασφαλίζεται ότι:

- κάθε φοιτητής παρακολουθεί το πολύ ένα μάθημα ανά ώρα.

A4. Διαλέξεις – Εργαστήρια & Χρονικές Εξαρτήσεις

Μεταβλητές Lecture / Lab

Κώδικας

```
Lecture[l,t] = model.NewBoolVar(...)
```

```
Lab[l,t] = model.NewBoolVar(...)
```

Κάθε μάθημα:

```
model.Add(sum(Lecture[l,t] for t in times) == 1)
```

Μαθήματα με εργαστήριο:

```
model.Add(sum(Lab[l,t] for t in times) == 1)
```

Διάλεξη πριν Εργαστήριο

Κώδικας

```
model.Add(lecture_time[l] < lab_time[l])
```

Ανάλυση

Οι χρονικές εξαρτήσεις:

- μειώνουν τον χώρο αναζήτησης,
- εισάγουν σειριακούς περιορισμούς,
- αυξάνουν τη δυσκολία επίλυσης.

Προαπαιτούμενα Μαθημάτων

Κώδικας

```
model.Add(lec_time[prereq] < lec_time[dependent])
```

Ανάλυση

Διασφαλίζεται ότι:

- μαθήματα με προαπαιτούμενα διδάσκονται με σωστή σειρά.

A5. Επίλυση & Έλεγχος

Κώδικας

```
solver = cp_model.CpSolver()  
solver.Solve(model)
```

Ανάλυση

Ο CP-SAT:

- συνδυάζει constraint propagation + search,
- είναι κατάλληλος για προβλήματα χρονοπρογραμματισμού,
- επιστρέφει εφικτή ή βέλτιστη λύση.

Η εκτύπωση:

- επαληθεύει την ορθότητα,
- επιτρέπει έλεγχο συγκρούσεων ανά φοιτητή.

```
---ΤΕΛΙΚΟ ΠΡΟΓΡΑΜΜΑ---  
  
Μάθημα : Math  
Ώρα : Mon_9, Wed_9  
Αίθουσα : R4  
Lecture: ['Mon_9']  
Lab: ---  
  
Μάθημα : Physics  
Ώρα : Mon_11, Tue_9  
Αίθουσα : R3  
Lecture: ['Mon_11']  
Lab: ---  
  
Μάθημα : Programming  
Ώρα : Mon_9, Tue_9  
Αίθουσα : R2  
Lecture: ['Tue_9']  
Lab: Tue_11  
  
Μάθημα : Networking  
Ώρα : Tue_11, Fri_9  
Αίθουσα : R2  
Lecture: ['Tue_11']  
Lab: Fri_9  
  
Μάθημα : CSP  
Ώρα : Tue_9, Thu_11  
Αίθουσα : R1  
Lecture: ['Thu_11']  
Lab: Fri_9
```

```
Μάθημα : Cryptography  
Ώρα : Wed_9, Fri_9  
Αίθουσα : R1  
Lecture: ['Fri_9']  
Lab: ---  
  
--- Ωράριο ανά φοιτητή + Έλεγχος Συγκρούσεων ---  
  
Φοιτητής: S1  
- Math: Mon_9, Room R4  
- Programming: Tue_9, Room R2  
- Networking: Tue_11, Room R2  
Χωρίς συγκρούσεις.  
  
Φοιτητής: S2  
- Physics: Mon_11, Room R3  
- Networking: Tue_11, Room R2  
Χωρίς συγκρούσεις.  
  
Φοιτητής: S3  
- Math: Mon_9, Room R4  
- Physics: Mon_11, Room R3  
- Networking: Tue_11, Room R2  
- CSP: Thu_11, Room R1  
Χωρίς συγκρούσεις.  
  
Φοιτητής: S4  
- Math: Mon_9, Room R4  
- Physics: Mon_11, Room R3  
- Programming: Tue_9, Room R2  
- CSP: Thu_11, Room R1  
- Cryptography: Fri_9, Room R1  
Χωρίς συγκρούσεις.
```

```
Φοιτητής: S5
- Math: Mon_9, Room R4
- Programming: Tue_9, Room R2
- CSP: Thu_11, Room R1
- Cryptography: Fri_9, Room R1
Χωρίς συγκρούσεις.

[Done] exited with code=0 in 15.826 seconds
```

Μέρος Β' – Ανάλυση Μοντελοποίησης

B1. Χρονικές Εξαρτήσεις

Οι χρονικές εξαρτήσεις αποτελούν έναν από τους σημαντικότερους παράγοντες αύξησης της δυσκολίας του προβλήματος χρονοπρογραμματισμού.

Επίδραση στη δυσκολία

Χωρίς χρονικές εξαρτήσεις, κάθε μάθημα μπορεί να τοποθετηθεί ανεξάρτητα σε οποιοδήποτε διαθέσιμο χρονικό slot. Στον κώδικα όμως, οι εξαρτήσεις:

- Προαπαιτούμενων μαθημάτων.
- Διαλέξεις πριν από εργαστήριο.

Εισάγουν μερική διάταξη μεταξύ μεταβλητών χρόνου.

Αυτό σημαίνει ότι ο solver δεν αναζητά πλέον οποιαδήποτε κατανομή, αλλά μόνο κατανομές που σέβονται συγκεκριμένη χρονική σειρά, περιορίζοντας τον χώρο λύσεων και αυξάνοντας το βάθος αναζήτησης.

Πως τροποποιούνται οι περιορισμοί

- Στο μοντέλο, κάθε μάθημα αποκτά ακέραια μεταβλητή χρόνου:

`lec_time[l]`

η οποία συνδέεται με τις δυαδικές μεταβλητές $x[l, t]$.

- Οι χρονικές εξαρτήσεις προαπαιτούμενων μαθημάτων υλοποιούνται ως:

`model.Add(lec_time[prereq] < lec_time[dependent])`

- Για τα εργαστήρια, ορίζονται επιπλέον ακέραιες μεταβλητές:

`lecture_time[l]`

`lab_time[l]`

και επιβάλλεται ο περιορισμός:

`model.Add(lecture_time[l] < lab_time[l])`

Έτσι ο solver διασφαλίζει όχι μόνο ότι διάλεξη και εργαστήριο δεν συμπίπτουν χρονικά, αλλά και ότι τηρείται η σωστή παιδαγωγική σειρά.

B2. Hard vs Soft Constraints

Hard constraints στο μοντέλο

Οι hard constraints είναι αυτοί που δεν επιτρέπεται να παραβιαστούν σε καμία εφικτή λύση.

Στον κώδικα περιλαμβάνουν:

- μοναδική ώρα ανά μάθημα: $\text{sum}(x[l, t]) == 1$
- μοναδική αίθουσα ανά μάθημα: $\text{sum}(y[l, r]) == 1$
- χωρητικότητα αιθουσών
- αποφυγή επικαλύψεων φοιτητών
- χρονικές εξαρτήσεις

Αυτοί οι περιορισμοί εκφράζουν φυσικούς ή ακαδημαϊκούς κανόνες που δεν μπορούν να παραβιαστούν.

Soft constraints στο μοντέλο

Οι soft constraints εκφράζουν προτιμήσεις ποιότητας και όχι απόλυτες απαιτήσεις.

Στον κώδικα, η ισορροπημένη χρήση αιθουσών αποτελεί soft constraint, καθώς δεν απαιτείται όλες οι αίθουσες να χρησιμοποιούνται ακριβώς το ίδιο, αλλά να αποφεύγεται η υπερφόρτωση μιας συγκεκριμένης αίθουσας.

Αυτό υλοποιείται με:

- μεταβλητές φόρτου αίθουσας `room_load[r]`
- μεταβλητές απόκλισης `dev[r]`
- αντικειμενική συνάρτηση ελαχιστοποίησης: `model.Minimize(sum(dev[r] for r in rooms))`

Έτσι ο solver επιλέγει τη λύση που ικανοποιεί όλους τους hard constraints και ταυτόχρονα βελτιστοποιεί την ποιότητα της κατανομής.

B3. Κενά και ομογενοποίηση προγράμματος

Τρέχουσα κατάσταση στο μοντέλο

Στο παρόν μοντέλο, τα κενά στο πρόγραμμα των φοιτητών αντιμετωπίζονται έμμεσα.

Ο περιορισμός: `model.Add(sum(x[l, t] for l in enrolled) <= 1)`

Αποτρέπει επικαλύψεις μαθημάτων, αλλά δεν λαμβάνει υπόψη τη συνοχή του προγράμματος.

Έτσι ένας φοιτητής μπορεί να έχει μαθήματα πρωί και απόγευμα με μεγάλα χρονικά κενά ενδιάμεσα.

Προτεινόμενη στρατηγική βελτίωσης

Για να ελαχιστοποιηθούν τα κενά, θα μπορούσαν να εισαχθούν:

- ακέραιες μεταβλητές που ορίζουν:
 - την πρώτη ώρα
 - την τελευταία ώρα ενός φοιτητή
- μεταβλητές που μετρούν τον αριθμό κενών χρονικών slots

Οι μεταβλητές αυτές θα προστίθενται ως όροι ποινής στην αντικειμενική συνάρτηση, παρόμοια με τον μηχανισμό των $\text{dev}[r]$.

Με αυτόν τον τρόπο, το μοντέλο θα προτιμούσε λύσεις με πιο <<συμπαγές>> πρόγραμμα, χωρίς να αποκλείει εφικτές λύσεις.

B4. Επιλογή Τεχνικής Επίλυσης

Αιτιολόγηση επιλογής CP-SAT

Το πρόβλημα περιλαμβάνει:

- δυαδικές μεταβλητές ανάθεσης (x, y, Lecture, Lab)
- ακέραιες μεταβλητές χρόνου
- λογικούς και χρονικούς περιορισμούς
- soft constraints με αντικειμενική συνάρτηση

Η χρήση Constraint Programming με CP-SAT solver επιτρέπει:

- ακριβή έκφραση όλων των περιορισμών
- εγγύηση εφικτής λύσης
- εύκολη ενσωμάτωση βελτιστοποίησης

Σύγκριση με άλλες τεχνικές

- ΕΞαντλητική αναζήτηση: μη εφαρμόσιμη λόγω εκθετικής αύξησης λύσεων
- Heuristics: γρήγορες αλλά χωρίς εγγύηση βέλτιστης λύσης
- Meta-heuristics: κατάλληλες για πολύ μεγάλα προβλήματα, αλλά απαιτούν πολύπλοκο tuning

Η επιλογή Constraint Programming με CP-SAT solver αποτελεί την καταλληλότερη προσέγγιση για το συγκεκριμένο πρόβλημα, καθώς προσφέρει ισορροπία μεταξύ εκφραστικότητας, αποδοτικότητας και εγγύησης ποιότητας λύσης.